Chapter 1

Introduction

1.1 Introduction

This chapter introduces the research problem and its primary motivation. It begins by discussing the growing complexity of modern software supply chains and the security challenges posed by the extensive use of open-source packages, third-party libraries, external dependencies, and interconnected software ecosystems [1],[2]. This chapter examines the limitations of traditional vulnerability detection approaches,emphasizing the need for more intelligent,adaptive,and context-aware analytical models capable of identifying unknown and zero-day vulnerabilities [3].Finally, it presents the research objectives and the overall structure of the thesis,providing a clear overview of this study.

Modern software systems increasingly rely on complex software supply chains, encompassing open-source packages, third-party libraries, external dependencies, versioned components, package repositories, and build pipeline.while these components facilitate faster application development and promote software reuse, they also introduce new security risks. These risk arise when vulnerable, outdated, or malicious components are reused across applications or inherited through direct and transitive dependencies [4].

Over the past decade, software supply chain security has emerged as a critical concern in cybersecurity research and practices. Attackers no longer solely target isolated applications or standalone source code components [5]. Instead, they increasingly focus on third-party libraries, package repositories, maintainer accounts, dependency chains, and build environments. These threats are challenging to detect because a component might appear secure in isolation; however, its true risk often becomes apparent only after analyzing its relationships, version history, and usage context [6].

Modern software ecosystems generate vast amounts of security-related data, including source code repositories, package metadata [7],dependency graphs, version histories, commit records, vulnerability reports, security advisories, and public vulnerability databases [8].Although these sources can help analysts understand software components and their associated risks, their sheer volume, diversity, and continuous render manual analysis increasingly challenging for security analysts, software maintainers and vulnerability management teams [9].

Traditional vulnerability detection approaches, such as static code analysis [10], rule-based scanning, signature-based detection, and known-vulnerabilities matching, remain valuable for identifying previously reported vulnerabilities [11]. However, these methods are often limited to addressing unknown or zero-day vulnerabilities that have not been publicly disclosed or recorded in databases [12].Furthermore,

many conventional approaches analyze software components in isolation, hindering the understanding of how risks propagate across the broader software ecosystem [13].

To overcome these limitations, cybersecurity research has increasingly explored data-driven approaches based on machine learning [14],[15]. Rather than relying solely on predefined signatures or manually designed rules, these approaches learn vulnerability-related patterns directly from software data.Graph-based techniques are particularly useful because software system naturally exhibit complex relationships among code elements, functions, packages, versions, and dependencies [16],[17],[18]. Previous studies have shown that representing software structures as graphs can effectively capture syntactic, semantic, control flow , and data flow relationships that are difficult to model using traditional feature-based approaches [20],[21].

Recent studies have extended this direction by exploring vulnerability paths [22], heterogeneous code relations [23],hierarchical graph refinement, and context-aware graph representation learning [24].These studies demonstrate the potential of graph learing for detecting complex vulnerability patterns within software systems. Most, however, primarily focus on source-code-level analysis [25], function-level classification [26],[27], or program-level vulnerability paths, rather than the broader supply chain-level risk propagation [28].

Despite this progress, a significant gap exits. Existing approaches do not fully model how security risks propagate across package dependencies, version relationships, repository interactions, and transitive dependency networks [29]. This gap is critical because software supply chain risks often arise from complex relationships among components, rather than merely from isolated code fragments [30].

Another challenge is the continuous evolution of software supply chains. New packages are released, dependencies are updated, vulnerabilities are disclosed, and attack techniques evolve over time; therefor, a component deemed safe at one moment may subsequently become a risk. Consequently, detection models cannot remain static; they require adaptive learning mechanisms that enable them to respond to new software components, dependency behaviors,and vulnerability patterns [31]. This adaptability is crucial for the early identification of indicators for unknown and potential zero-day vulnerabilities [32],[33].

Interpretability is crucial for practical vulnerability detection [34]. Security analysts and software maintainers require a clear understanding of why a specific component, dependency path, package version, or repository is deemed to be high risk. Without sufficient explanation, purely machine learning based models produce predictions that limit their utility in real-world vulnerability management, prioritization, and remediation planning [35].

Motivated by these challenges, this study proposes a graph-based adaptive machine learning framework to identify unknown and zero-day vulnerabilities in software supply chains.This framework integrates several key components:software supply chain data collection,dependency graph construction,feature representation learning, graph-based vulnerability detection, adaptive risk inference, and explainable security output generation [36],[37].By combining graph-structured representation learning with adaptive machine learning, this study aims to support early vulnerability detection, risk prioritization, dependency-level security analysis, and explainable decision-making in software supply chain security [38].

static code analysis and rule-based detection systems,which examine source code without execution or rely on manually defined rules [39], face distinct limitations. Although capable of detecting well-understood vulnerability patterns, they often require continuous updates and can produce false positives or miss vulnerabilities dependent on execution context, complex program behavior, or external dependency relationships.

Signature-based detection and known vulnerability matching encounter similar issues [40]. While effective when vulnerabilities are already discovered, documented, and mapped to specific components or versions, these methods are less effective against emerging threats. Unknown or zero-day vulnerabilities lack established signatures or database records, a critical limitation in software supply chains where risks may manifest before formal reporting or accurate linkage to affected packages.

A further significant limitation is that many traditional approaches analyze software components in isolation.They can identify vulnerabilities within a file, function, package, or version, but often fail to capture how the risk propagates across dependency chains [41]. In software supply chains, the impact of a vulnerable component extends beyond the component itself, encompassing its relationship with other packages,versions, repositories, and downstream applications [42].

One primary challenge is the sheer complexity of dependency relationships.Software projects frequently incorporate not only direct dependencies chosen by developers but also numerous transitive dependencies introduced through other packages [43].These sprawling dependency networks can become exceptionally large and difficult to trace. Consequently, precisely identifying which component introduces a risk and how that risk propagates to affect downstream applications is a formidable task.

Vulnerability propagation presents another significant challenge. A single vulnerable package, when widely reused across various projects, can affects numerous applications. Its actual impact often hinges on its specific position within the dependency graph and the inheritance paths it traverses. This intricate propagation makes supply chain Vulnerability detection considerably more complex than isolated, code-level analysis[44].

Finally, data quality and availability constitute a substantial challenge. Software security data is frequently heterogeneous, incomplete, delayed, or inconsistent across various sources [45]. Public vulnerability databases, for instance, often lack comprehensive listings of all affected versions, and available databases may be limited or imbalanced. Such deficiencies significantly impede model training and evaluation, particularly for unknown or previously unobserved availabilities.

Interpretability remains a significant challenge in machine learning-based availability detection. Security analysts and software maintainers require clear explanations to understand why a component, package version, or dependency path is considered high-risk.Systems that produce opaque predictions without meaningful explanations are difficult to trust and deploy effectively in real-world availability management.

Addressing this challenge necessitates detection approaches that are relationship aware ,adaptive,and explainable. The framework proposed in this thesis aims to meet these requirements by modeling software supply chains as graph-structured data. It learns vulnerability-related patterns across dependency relationships, adapts to newly emerging risks, and generates interpretable outputs to support risk prioritization and decision-making.

1.2 Research Problem

Modern software supply chains are complex, highly interconnected ecosystems comprising open-source packages, third-party libraries, versioned components, repositories, and build processes [46]. Detecting unknown and zero-day vulnerabilities within these ecosystems presents a significant challenge, particularly when risks are hidden within direct or transitive dependency relationships [1],[5].

While existing vulnerability detection approaches effectively identify many known vulnerabilities, they often rely on static analysis, predefined rules, signature-based matching, or publicly reported vulnerability records [3],[12],[47]. Consequently, they offer limited support for the early detection of vulnerabilities that have not yet been disclosed, labeled , or recorded in vulnerability databases.

A further critical limitation is the weak modeling of dependency relationships. Many current methods analyze software components in isolation,failing to fully capture how risk propagates across packages, versions,repositories, and downstream applications [6],[13],[48]. In software supply chain, the impact of a vulnerable component often depends on its position within the dependency network and its relationships with other components.

Therefore, there is a clear need for a vulnerability detection approach capable of analyzing software components in conjunction with their dependency relationships [12],[5]. Such an approach must infer suspicious patterns from structural and contextual information, adapt to evolving software ecosystems, and provide interpretable outputs that facilitate security decision-making.

This research investigates the design of a graph-based adaptive machine learning framework for identifying unknown and zero-day vulnerabilities in software supply chains.The proposed framework specifically aims to support dependency-aware detection, adaptive risk analysis, and explainable vulnerability prioritization.

1.3 Purpose Statement

The purpose of this study is to design and evaluate a graph-based adaptive machine learning framework for identifying unknown and zero-day vulnerabilities in software supply chains. The study aims to analyze software components together with their dependency relationships, learn vulnerability-related patterns from graph-structured data, adapt to evolving software ecosystems, and generate explainable outputs that support risk prioritization and security decision-making [].

1.4 Research Objectives

The main objective of this research is to design and develop a graph-based adaptive machine learning framework for identifying unknown and zero-day vulnerabilities in software supply chains.To achieve this, the specific objectives are:

1.5 sub-Objectives

- Data and Graph Construction: To design a pipeline for collecting and integrating software supply chain data, and to construct a graph-based representation of packages, dependencies, and vulnerability indicators as interconnected nodes and edges. - Pattern Learning and Detection: To develop graph-based learning techniques for capturing structural patterns from dependency graph and identifying suspicious indicators of unknown or zero-day vulnerabilities.

- Adaptive Learning: To incorporate adaptive mechanisms that update detection knowledge as new

packages, dependencies, and vulnerability information emerge.

- Explainability and Evaluation: To generate interpretable, risk-oriented outputs for prioritization and to evaluate the framework's effectiveness using vulnerability-related datasets.

1.6 Main Research Questions

How can a graph-based adaptive machine learning framework be designed to identify unknown and zero-day vulnerability in software supply chains by analyzing software components and their dependency relationships?

This study addresses the following research questions, derived from the research problem and objectives:

RQ1: How can data from software supply chains such as packages, versions, repositories, dependencies, and vulnerability reports be collected and organized for vulnerability detection?

RQ2: How can software supply chains be represented as graphs to show the relationships between software components and their dependencies?

RQ3: How can graph-based machine learning be used to detect suspicious dependency patterns that many indicate potential unknown or zero-day vulnerabilities?

RQ4: How can the proposed model adapt when new packages,versions,dependencies, or vulnerability information appear?

RQ5: How can the proposed framework provide clear explanations to help security analysts understand and prioritize high-risk components or dependency paths?

1.7 Research Hypotheses

H1: A graph-based adaptive machine learning framework will improve the identification of high-risk software components components compared with approaches that analyze components in isolation.

H2: Modeling software supply chains as dependency graphs will improve the ability to detect suspicious dependency structures and potential vulnerability propagation path.

H3: Incorporating adaptive learning mechanisms will improve the framework's ability to respond to newly emerging packages, dependency changes, and vulnerability information.

H4: Providing explainable outputs will improve the practical usefulness of the framework for vulnerability prioritization and security decision-making.

1.8 Research Significance

The rapid expansion of software supply chains and open-source ecosystems has dramatically increased the complexity of software security. Modern applications rely on numerous external packages, third-party libraries, versioned components, and transitive dependencies. While these interconnected components enhance software reuse and accelerate development, they also introduce significant security risks that traditional vulnerability detection methods struggle to address.

Traditional vulnerability detection approaches frequently fail to identify unknown and zero-day vulnerabilities, which remain undisclosed or unrecorded in public databases. Many existing methods depend on static rules,known vulnerabilities records, or isolated code analysis. Consequently, there is a pressing need for intelligent vulnerability detection approaches that can analyze complex dependency relationships, identify suspicious patterns, and facilitate early risk discovery across software supply chains.

This research is significant for its development of a graph-based adaptive machine learning framework specifically designed for software supply chain vulnerability detection. The proposed framework models software components, versions, repositories, dependencies, and vulnerability indicators as graph-structured data. This approach enables comprehensive analysis of individual components and the dependency paths through which security risks can propagate.

Another key contribution is the framework's use of adaptive learning mechanisms. Software ecosystems are dynamic, with new packages, updated dependencies, and emerging vulnerability information constantly appearing. By incorporating adaptive learning, the framework can continuously update its detection knowledge, thereby improving its ability to identify emerging risks and potential zero-day vulnerability indicators over time.

Furthermore, the proposed framework supports explainable vulnerability analysis. Security analysts and software maintainers require more than a simple ”vulnerable” or ”non-vulnerable” verdict; they need to understand why a component, version, or dependency path is deemed high-risk. Providing explainable, risk-oriented outputs will facilitate vulnerability prioritization, remediation planning, and informed security decision-making.

Overall, this research advances proactive and context-aware vulnerability detection by shifting the analytical focus from isolated components to a dependency-aware software supply chains perspective. By integrating graph-based representation, adaptive learning, and explainable outputs, the proposes framework aims to significantly improve early vulnerability detection, adaptive risk analysis, and explainable vulnerability prioritization.

1.9 Scope of the Thesis

This thesis presents a graph-based adaptive machine learning framework designed to identify unknown and zero-day vulnerabilities within software supply chains.

Its primary objective is to detect suspicions dependency patterns and high-risk software components that may signal such vulnerabilities. To achieve this, the framework analyzes software components and their complex dependency relationships-including packages, versions, repositories, dependency files, and vulnerability indicators-employing graph-based learning techniques capable of modeling relationships among software entities and detecting risk propagation across dependency networks.

The research scope encompasses the development of a data processing pipeline that collects and integrates software supply chains data from diverse sources. These include code repositories, package metadata, dependency files, vulnerability reports, security advisories, and public vulnerability databases. This collected data is then transformed into graph-structured representations suitable for vulnerability detection and risk analysis.

Furthermore, the framework inncorporates adaptive learning mechanisms. These mechanisms enable it to continuously update its detection knowledge as new packages, versions, dependencies, or vulnerability reports become available. It also generates explainable, risk-oriented outputs, assisting security analysts and software maintainers in understanding and prioritizing high-risk components or dependency paths.

1.10 Limitations of the Study:

This study is subject to several limitations. First, the framework depends on the availability and quality of software supply chains and vulnerability-related data. Public vulnerability databases and security advisories may be incomplete, dalayed, or inconsistent, particularly for newly emerging or undisclosed vulnerabilities.

Second, the study aims to identify suspicious indicators of unknown and potential zero-day vulnerability risk; however, it does not guarantee the discovery of all zero-day vulnerability. Such vulnerabilities may lack public records, labels, or sufficient historical examples for model training and validation.

Third, the evaluation of the framework will depend on selected vulnerability-related datasets and software dependency data. Therefore, the findings may vary when applied to different ecosystems, package repositories, programming languages, or real-world deployment environments.

Fourth, the proposed framework focuses on vulnerability detection and risk prioritization in software supply chains. It does not include exploit development, malware reverse engineering, automated patch generation, real-time attack prevention, or full incident response workflows.

1.11 Delimitation of the thesis:

It is important to clarify the boundaries of this thesis. While it focuses on software supply chains vulnerability detection and risk analysis, it does not guarantee the discovery of all zero-day vulnerabilities. The research also excludes exploit development, malware analysis, real-time attack prevention, or full incident response workflows.

The framework's effectiveness is evaluated using vulnerability-related datasets and software dependency data, such as Public vulnerability records, package metadata, and dependency graphs. This evaluation assesses its ability to detect suspicious dependency graphs. This evaluation assesses its ability to detect suspicious dependency structures, prioritize high-risk components, and provide interpretable vulnerability analysis results.

1.12 Terminology Definitions

Software Supply Chain: A software ecosystem consisting of packages, libraries, repositories, versions, dependencies, build processes, and related metadata that collectively support software development and deployment.

Dependency Graph: A graph-based representation in which software components are modeled as nodes and their dependency relationships are modeled as edges to support relationship-aware analysis.

Direct Dependency: A package or library explicitly required by a software project.

Transitive Dependency: A package or library introduced indirectly through another dependency, which may affect the security posture of the main software project.

Unknown Vulnerability: A vulnerability that has not yet been publicly documented, labeled, or linked to an established vulnerability record.

Zero-day vulnerabilities: A vulnerability that is unknown to vendors or security communities at the time it may be exploited, and for which no public patch or confirmed disclosure may yet exist.

Graph-based Machine Learning: A learning approach that uses graph structures to capture relationships among software entities and learn vulnerability-related patterns from connected data.

Adaptive Learning: A learning mechanism that updates detection knowledge as new packages, versions, Dependencies, vulnerability reports, or behavioral patterns become available.

Explainability: The ability of a model to provide interpretable outputs that clarify the factors, relationships, or patterns contributing to a vulnerability or risk prediction for a component, package version, repository, or dependency path.

1.13 Main Contributions :

This research introduces a novel graph-based adaptive machine learning framework that addressees the critical problem od identifying unknown and zero-day vulnerabilities in software supply chain. It shifts vulnerability detection from isolated component analysis to a more dependency-aware, adaptive, and explainable supply chain risk analysis approach. The primary contributions include:

1.13.1 Dependency-Aware vulnerability Detection :

This thesis moves beyond isolated analysis by introducing a detection perspective that analyzes software components within their broader dependency context. It examines how risk emerges and propagates through connections across the software supply chain, rather than treating packages, versions, and repositories as standalone units.

1.13.2 Unified Graph Modeling of Software Supply Chains :

The proposed approach unifies the representation of software supply chain by modeling them as graph-structure data. This strategy effectively captures software entities, dependency links, version relationships, and vulnerability indicators for comprehensive risk analysis.

1.13.3 Graph-Based Learning for Suspicious Pattern Identification :

This research leverages graph-based machine learning to identify structural and contextual patterns within software supply chain graphs. This capability enables the detection of suspicious dependency structures indicative of emerging, unknown, or potential zero-day vulnerability risks.

1.13.4 Adaptive Detection for Evolving Software Ecosystems:

The framework incorporates adaptive learning mechanisms to continuously update its detection knowledge. This ensures the framework remains effective in dynamically evolving software ecosystems by integrating new packages, versions, dependencies, or vulnerability reports as they become available.

The framework incorporates adaptive mechanisms that update detection knowledge as packages, versions, dependencies, and vulnerability information evolve. This adaptation addresses the dynamic nature of software ecosystems and supports earlier recognition of emerging risks.

1.13.5 Explainable Risk Prioritization:

The framework generates interpretable outputs, clarifying why component, version, or dependency path is deemed high-risk. This feature assists security analysts and software maintainers in prioritizing remediation actions and making informed security decisions.

1.13.6 Experimental Evaluation of Supply Chains:

This thesis evaluates the framework using vulnerability-related datasets and software dependency data. The evaluation assesses its ability to detect suspicious dependency structures, prioritize high-risk components, and produce interpretable vulnerability analysis results.

Collectively, these contributions support the thesis's main aim:to develop a proactive, dependency-aware, adaptive, and explainable approach for identifying unknown and zero-day vulnerabilities risks in software supply chains.

1.14 Thesis Organization :

The remainder of this thesis is organized as follows:

Chapter 2 reviews relevant literature on software vulnerability detection, supply chains security, graph-based machine learning, adaptive vulnerability detection, and explainable security analysis. It also provides a comparative analysis of previous studies and identifies the research gap addressed by this work.

Chapter 3 presents the research methodology and the proposed framework. It describes the overall graph-based adaptive machine learning architecture, data collection process, prepossessing steps, software supply chains graph construction, feature representation, the graph-based vulnerability detection model, adaptive learning mechanism, and explainable output generation.

Chapter 4 details the implementation and experimental setup of the proposed framework. It discusses the software supply chain data used, including public vulnerability-related datasets. The chapter also explains the graph construction process, model configuration, training and testing procedures, and evaluation metrics used to assess detection and prioritization performance.

Chapter 5 presents the experimental results and discussion. It evaluates the proposed framework's ability to identify suspicious dependency structures, detect high-risk software components, prioritize vulnerability risks, and generate interpretable analysis results. The chapter also discusses these findings in relation to the research objectives and highlights the limitations of the proposed approach.

Chapter 6 presents the conclusions, limitations, recommendations, and future work. It summarizes the study's main findings and contributions, discusses the proposed framework's limitations, and outlines future directions. These include improving adaptive learning, expanding dependency data source, enhancing explainability, and evaluating the framework using larger real-world software supply chain datasets.

Chapter 2

Literature Review

2.1 Introduction

This chapter reviews existing literature on software vulnerability detection, graph-based learning, and software supply chain security. The purpose of this paper is to provide a comprehensive overview of previous research and technological approaches to addressing the challenges of detecting unknown and zero-day vulnerabilities in modern software ecosystems.

The chapter discusses traditional vulnerability detection techniques, machine learning-based detection methods, and recent developments in graph-based vulnerability analysis. This review establishes the theoretical foundation for the proposed graph-based adaptive machine learning model and identifies the research gap addressed by this study.

2.2 Background and Definitions

2.2.1 Types of Vulnerabilities

Software vulnerabilities [3] are weaknesses in software systems that attackers may exploit to compromise security. These vulnerabilities can stem from programming errors, improper input handling, insecure dependencies, or flaws in software design. Common categories discussed in vulnerability detection research include memory-related vulnerabilities [49], input validation vulnerabilities [50], software supply chain vulnerabilities [5], and blockchain-based smart contract vulnerabilities [51].

- **Memory-Related Vulnerabilities:** Memory-related vulnerabilities [10] represent one of the most critical classes of software weaknesses. They often arise from improper memory management, such as writing data beyond allocated boundaries or accessing memory after it has been freed. Buffer overflows [26], a common example, allow attackers to exploit memory corruption to manipulate the program behavior or execution flow.
- **Input Validation Vulnerabilities:** Input validation vulnerabilities [39] arise when software fails to properly validate external inputs. These weaknesses can allow attackers to inject malicious data, manipulate program logic, or execute unintended commands. Examples include injection attacks [12] and other vulnerabilities resulting from insufficient user-supplied data checking.

- **Software Supply Chain Vulnerabilities:** Software supply chain vulnerabilities [5] are increasingly critical due to the growing reliance on third-party libraries, open-source packages, and external dependencies [9]. Here, a vulnerability might not reside directly in the main application code but instead propagate through dependencies or compromised software components [13].
- **Blockchain and Smart Contract Vulnerabilities:** Blockchain and smart contract vulnerabilities constitute an emerging category of security weaknesses. These often stem from logical flaws in smart contract execution, such as reentrancy, integer overflow, unchecked calls, and access control weaknesses [51]. Given their operation in decentralized environments, these vulnerabilities can lead to serious financial and security consequences.

2.2.2 Vulnerability Detection Methods

Vulnerability detection methods aim to identify security weaknesses in software before they are exploited. Prior to the adoption of machine learning and deep learning approaches, research in this area primarily relied on traditional program analysis techniques, including static analysis [39], dynamic analysis , and fuzz testing [3]. These methods have long served as foundational approaches in software security by enabling systematic identification of vulnerabilities across source code, binary code, and runtime behavior.

1. **Static Analysis:** This technique examines source code or binary code without executing the program, enabling early detection of potential vulnerabilities during the development phase. It typically employs techniques such as pattern matching, taint analysis, and control-flow and data-flow inspection. Despite its usefulness in early-stage vulnerability detection, static analysis is often limited by high false-positive rates and its inability to capture runtime behavior and complex semantic interactions within software systems.
2. **Dynamic Analysis:** In contrast, dynamic analysis observes program behavior during execution. This approach is particularly effective in detecting runtime errors,memory corruption issues, and environment-dependent vulnerabilities that may not be observable through static inspection. However, its effectiveness is strongly dependent on the quality and coverage of test inputs, meaning that unexecuted code paths may still contain hidden vulnerabilities.
3. **Fuzz Testing:** Fuzz testing is a widely used complementary technique that generates large volumes of random or malformed inputs to trigger unexpected program behavior. It is particularly effective in uncovering memory-related vulnerabilities [3] such as buffer overflows and input-handling errors [26]. Nevertheless, fuzzing is limited in its ability to detect deep logical vulnerabilities that require specific program states or complex execution sequences.

Despite their importance, traditional vulnerability detection techniques face inherent limitations when applied to modern large-scale and complex software systems. Static analysis often produces a high number of false positives, dynamic analysis suffers from incomplete execution coverage, and fuzz testing is computationally expensive while heavily dependent on input generation strategies. More importantly, these methods rely on manually defined rules and known vulnerability patterns, which restricts their ability to detect previously unknown or zero-day vulnerabilities.

Table 2.1: Comparison of Traditional Vulnerability Detection Methods.

Method	Representative Studies	Strengths	Limitations
Static Analysis	Chess & West [1], Yamaguchi et al. [6]	Detects vulnerabilities early without executing the program	High false-positive rate and limited runtime context
Dynamic Analysis	Shoshitaishvili et al. [2]	Observes real program behavior during execution	Limited coverage depending on test scenarios
Fuzz Testing	Sutton et al. [3]	Effective in discovering memory-related vulnerabilities	Highly dependent on input generation strategies

As highlighted in Table 1, each traditional technique offers important security benefits but also presents specific limitations. Static analysis supports early detection but often lacks runtime awareness. Dynamic analysis improves runtime visibility but depends on execution coverage. Fuzz testing excels at discovering memory-related flaws but is sensitive to input generation strategies. Consequently, traditional techniques are increasingly being complemented by learning-based methods capable of capturing deeper structural and semantic relationships in software.

2.2.3 Graph-Based Vulnerability Detection

Graph-based learning techniques [17],[23] have recently emerged as a promising direction for vulnerability detection, enabling the modeling of complex structural relationships within source code that traditional methods often fail to capture. By representing programs as graphs [16] that encode syntactic and semantic dependencies, these approaches allow machine learning models to analyze software behavior more effectively.

Early research in graph-based vulnerability detection primarily focused on applying graph neural networks (GNNs) [23] to analyze these structural relationships within source code.

For instance, Zhou et al. introduced Devign [17], one of the earliest frameworks to apply GNNs for automated vulnerability detection in source code. Devign addressed the shortcomings of traditional detection techniques by modeling source code as program graphs that capture both syntactic and semantic dependencies. This structural analysis of software behavior enabled superior classification of vulnerable functions compared to conventional machine learning and static analysis methods. Its evaluation on real-world datasets from open-source projects like the Linux Kernel, FFmpeg, QEMU, and Wireshark underscored its practical relevance, establishing Devign as a seminal contribution to software security analysis.

However, Devign exhibits notable limitations. It primarily focuses on vulnerability classification without offering explainable localization, its deep GNN architecture introduces significant computational complexity with over one million trainable parameters , and it does not extend to modeling dependencies across software supply chain environments. Consequently, while Devign represents a foundational step, subsequent research has focused on improving interpretability and vulnerability localization [20] within graph-based security analysis frameworks.

To improve vulnerability detection accuracy, subsequent research has explored more expressive graph representations [16] capable of modeling diverse program relationships.

Luo et al. introduced a heterogeneous graph neural network-based framework for vulnerability detection. This framework addresses the limitations of homogeneous graph representations [23], which treat all program elements uniformly, and the shortcomings of traditional models that struggle to capture diverse structural relationships within complex software. By modeling source code using Inter-Procedural Abstract Graphs (IPAGs) that integrate syntactic, semantic, and inter-procedural dependencies such as function calls and data flows, the framework enables a richer structural representation of program behavior. A Heterogeneous Attention Graph Neural Network (HAGNN) is employed to learn multiple dependency patterns across these heterogeneous graph components, thereby improving vulnerability classification performance. Evaluated on datasets from the National Vulnerability Database (NVD) [53] and the Software Assurance Reference Dataset (SARD), which together covered over one hundred vulnerability categories, the framework demonstrated strong detection performance, achieving an accuracy of 96.6% for C programs and 97.8% for Java programs.These findings highlight the effectiveness of heterogeneous graph modeling for vulnerability detection in complex software systems. Nevertheless, the framework remains limited to single-program vulnerability analysis and does not explicitly model vulnerability propagation across software supply chain environments, restricting its applicability to modern interconnected software ecosystems. Thus, while heterogeneous graph modeling improves structural vulnerability representation, further research must extend such approaches toward dependency-aware models capable of analyzing vulnerability propagation across software supply chains.

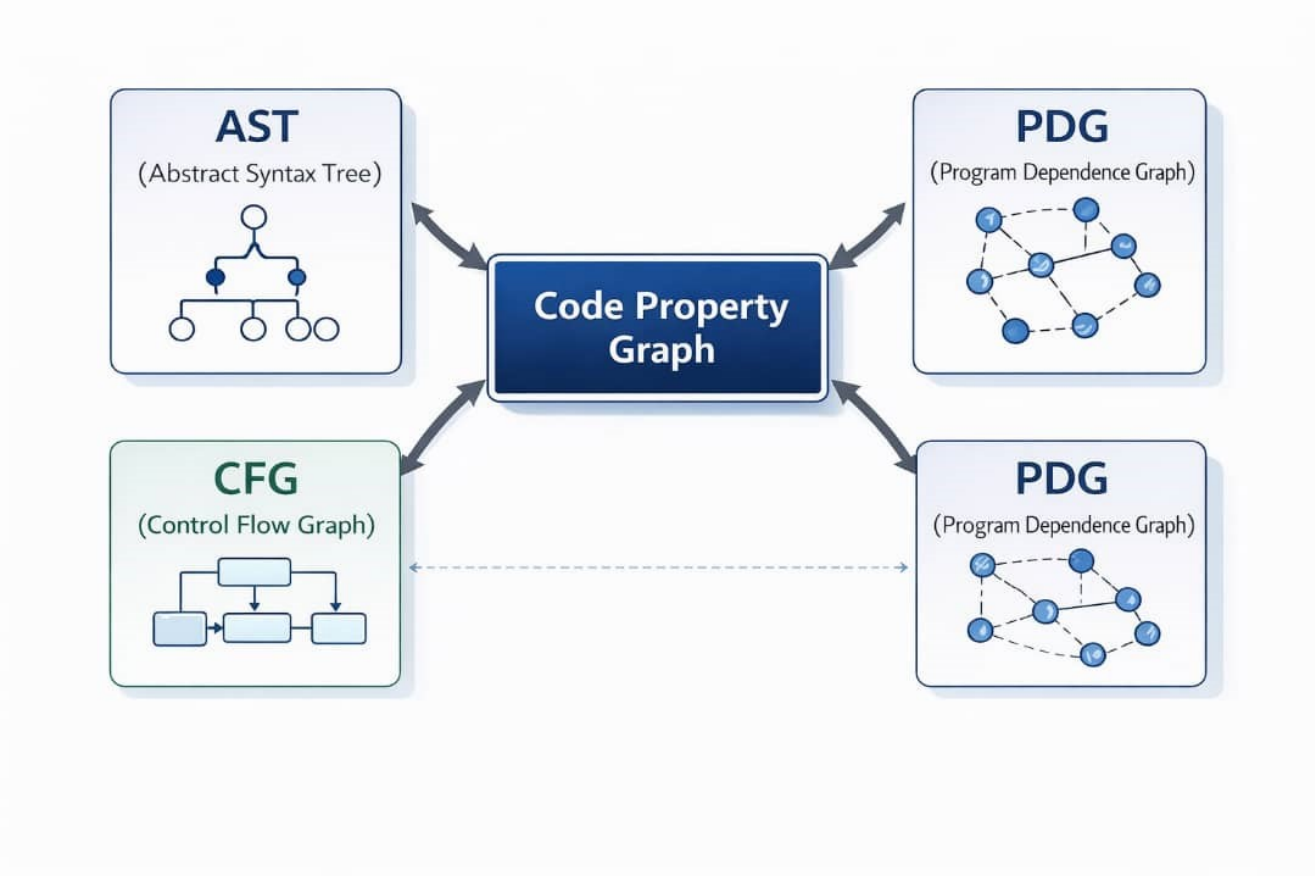
Peng et al. [18] introduced ANGEL, a hierarchical and context-aware graph-based framework designed to enhance vulnerability detection in large and structurally complex code graphs. This approach addresses limitations of conventional static analysis and deep learning techniques, which often fail to effectively capture long-range semantic dependencies and struggle with scalability in large software systems. ANGEL models source code using Code Property Graphs (CPGs), unifying Abstract Syntax Trees (AST), Control Flow Graphs (CFG), and Program Dependency Graphs (PDG) to comprehensively represent both syntactic and semantic program relationships. It integrates Graph Neural Networks (GNNs) with Graph Transformer architectures to capture both local structural dependencies and global contextual interactions within software programs. To mitigate redundancy and scalability issues in large code graphs, the framework incorporates a hierarchical graph refinement mechanism that compresses graph structures while preserving vulnerability-relevant information.

The framework was evaluated on widely used benchmark datasets, including Devign [17], Reveal, and Big-Vul [26], which contain vulnerability samples derived from CVE repositories [7]. Experimental evaluation demonstrates that ANGEL outperforms several state-of-the-art approaches, including token-based models (e.g., VulDeePecker [11] and SySeVR) and graph-based frameworks (e.g., Devign, IVDetect, DeepWukong, and MAGNET). These findings highlight the effectiveness of hierarchical graph learning for scalable vulnerability detection in large and structurally complex software systems. However, the framework primarily focuses on intra-program vulnerability detection and does not explicitly model dependency relationships across software ecosystems or supply chain environments, which limits its applicability to modern interconnected software systems. Therefore, while ANGEL significantly improves scalability and long-range dependency modeling in graph-based vulnerability detection, further research is needed to extend such architectures toward dependency-aware security models capable of analyzing vulnerability propagation across software supply chains [9].

Table 2.2: COMPARATIVE SUMMARY OF REPRESENTATIVE GRAPH-BASED VULNERABILITY DETECTION FRAMEWORKS.

Framework	Graph Representation	Dataset	Strengths	Limitations
Devign (Zhou et al.) [17]	Program Graph / Code Property Graph encoding syntactic and semantic relationships	Devign dataset derived from Linux Kernel, FFmpeg, QEMU, Wireshark	One of the earliest frameworks applying GNNs for vulnerability detection; effectively captures semantic dependencies in source code; improves classification accuracy over traditional ML and static analysis methods	Lacks explainable vulnerability localization; high computational complexity (>1M parameters); does not model software supply chain dependencies
HAGNN (Luo et al.) [23]	Inter-Procedural Abstract Graph (IPAG) with heterogeneous graph structure	NVD + SARD datasets (100+ vulnerability types)	Improves vulnerability classification using attention-based graph learning; high detection accuracy (96.6% for C, 97.8% for Java)	Limited to intra-program analysis; does not model vulnerability propagation across software ecosystems
ANGEL (Peng et al.) [18]	Hierarchical Code Property Graph (AST + CFG + PDG) combined with Graph Transformer	Devign, Reveal, Big-Vul datasets derived from CVE repositories	Captures long-range semantic dependencies; improves scalability for large code graphs; outperforms several state-of-the-art models (Devign, IVDetect, DeepWukong, MAGNET)	Focuses mainly on intra-program vulnerability detection; does not explicitly consider supply chain dependency relationships

FIGURE 1. OVERVIEW OF GRAPH REPRESENTATIONS (AST, CFG, PDG, CPG)



This figure illustrates the relationships among graph representations and shows how the Code Property Graph (CPG) integrates AST, CFG, and PDG.

2.2.4 Explainable Graph-Based Vulnerability Detection

Atashin et al. introduced VulPathFinder [53], a graph-based framework designed for explainable vulnerability path discovery in open-source software systems. This approach addresses a critical limitation of conventional vulnerability detection techniques, which typically identify vulnerable code segments without providing insight into the underlying causes of security flaws. VulPathFinder models source code using Code Property Graphs (CPGs) [16] that integrate control-flow and data-flow dependencies, enabling comprehensive structural analysis of program behavior and facilitating the identification of potential sink points responsible for security weaknesses. A Graph Neural Network (GNN) is employed to detect candidate vulnerability sinks, followed by backward program slicing to reconstruct potential vulnerability execution paths and reveal the causal relationships leading to security flaws.

The framework was evaluated on the Software Assurance Reference Dataset (SARD), which contains a wide range of vulnerability instances, including memory-related weaknesses like buffer overflows. Experimental results demonstrate strong vulnerability detection capabilities, achieving an F1-score of 0.98 and an explanation accuracy of approximately 98%, measured using Triggering Line Coverage metrics. These findings highlight the effectiveness of explainable graph-based analysis for vulnerability localization and demonstrate the potential of graph-based learning techniques for improving transparency in automated vulnerability detection systems.

However, VulPathFinder remains limited to intra-program vulnerability analysis, meaning vulnerabilities are analyzed only within the boundaries of individual software programs. Consequently, the approach does not explicitly consider dependency relationships across software ecosystems or supply chain environments, which restricts its applicability to complex, interconnected software systems. Therefore, while VulPathFinder advances explainable graph-based vulnerability analysis, further research is required to extend such approaches toward dependency-aware security models capable of analyzing vulnerability risks across complex software supply chains.

Compared to traditional techniques that primarily focus on identifying vulnerable code segments, VulPathFinder represents an early attempt at providing explainable vulnerability localization [53] through graph-based program representations. However, subsequent research has explored more advanced explanation mechanisms, integrating causal reasoning and representation learning to further improve both interpretability and robustness in graph-based vulnerability detection models .

Understanding the structural causes that influence model predictions in graph-based vulnerability detection remains a challenging problem, even with significant improvements from causal explanation frameworks . Consequently, recent studies have explored counterfactual reasoning techniques [35] to provide more precise explanations of vulnerability detection decisions.

In this context, Cao et al. introduced COCA, a contrastive causal explanation framework designed to enhance the interpretability and robustness of Graph Neural Network (GNN)-based vulnerability detection models. While recent graph-based vulnerability detection frameworks achieve strong performance, they often operate as black-box models [34], making it difficult to interpret their decisions or pinpoint vulnerability-inducing code patterns within complex program graphs [14]. COCA addresses this critical limitation by providing precise explanations and structural localization.

The framework represents source code using graph-based program representations, such as Code Property Graphs (CPGs) and Program Dependence Graphs (PDGs) [16]. This allows it to capture essential structural relationships, including control-flow and data-flow dependencies, which are critical for accurate vulnerability analysis. COCA's architecture comprises two main components: COCA-Tra, which improves the robustness of vulnerability detectors through contrastive representation learning and semantic-preserving code transformations; and COCAExp, an explanation module that em-

ploys dual-view causal reasoning by combining factual and counterfactual analysis to identify minimal vulnerability-relevant subgraphs responsible for the model's predictions.

COCA was evaluated on several widely used vulnerability detection benchmarks, including Devign [17], Reveal, BigVul, CrossVul, and CVEfixes [34], which contain large collections of real-world vulnerable and non-vulnerable code functions. Experimental results demonstrate that COCA improves both vulnerability detection performance and explanation quality compared to existing explainability approaches like GNNExplainer, PGExplainer, and CFExplainer. By extracting concise program subgraphs that highlight vulnerability-inducing statements, the framework enables more transparent vulnerability analysis and assists developers in identifying the root causes of software vulnerabilities.

Despite its strengths, the framework exhibits several limitations. First, its explanation mechanism primarily focuses on intra-program vulnerability analysis, detecting vulnerabilities within individual software systems, and thus does not explicitly capture dependency relationships across software ecosystems or software supply chain environments. Second, COCA relies on graph representations derived from individual program functions, which limits its ability to model vulnerability propagation across interdependent software components. Additionally, generating causal explanations for large program graphs may introduce significant computational overhead, potentially affecting scalability when applied to large-scale software systems.

Therefore, while COCA represents an important step toward interpretable graph-based vulnerability detection, future research must advance toward scalable, dependency-aware security models capable of analyzing vulnerability propagation across complex software supply chains.

Graph neural network (GNN)–based vulnerability detection models have achieved substantial progress; however, their limited interpretability remains a critical challenge in practical software security analysis. While recent deep learning approaches effectively capture structural and semantic patterns in source code, their predictions often lack transparency, preventing developers from understanding the root causes of detected vulnerabilities. To address this, Chu et al. introduced CFExplainer [35], one of the first frameworks to formulate vulnerability explanation as a counterfactual reasoning problem within graph-based vulnerability detection models. By representing source code as program graphs and leveraging differentiable edge masking, CFExplainer identifies minimal structural perturbations that alter a model's prediction outcome, thereby revealing the critical subgraphs responsible for vulnerability detection. This counterfactual explanation mechanism facilitates more precise localization of vulnerability-inducing code patterns and enhances the transparency of GNN-based detection systems.

CFExplainer was evaluated on the widely used Big-Vul dataset [26], a large-scale vulnerability dataset derived from real-world CVE reports and open-source software repositories. Experimental results showed that CFExplainer consistently outperforms several state-of-the-art explanation techniques—including GNNExplainer, PGExplainer, SubgraphX, GNN-LRP, DeepLIFT, and Grad-CAM—across multiple vulnerability-oriented and model-oriented evaluation metrics. These findings underscore the effectiveness of counterfactual reasoning in accurately localizing vulnerability-inducing code patterns and significantly improving the interpretability of graph-based vulnerability detection models.

Despite its strengths, CFExplainer has several important limitations. First, it primarily focuses on intra-program vulnerability analysis, deriving explanations from structural perturbations within individual program graphs. Second, the approach does not capture dependency relationships across software ecosystems or analyze vulnerability propagation across interconnected software components. Finally, the framework relies on existing vulnerability detection models, focusing on explaining their decisions rather than advancing the detection capability itself.

Thus, while CFExplainer represents a significant step toward explainable graph-based vulnerability

detection, further research is needed to develop scalable and dependency-aware security models capable of analyzing vulnerability propagation across complex software supply chains. Moreover, despite the interpretability benefits of counterfactual explanation techniques, challenges related to robustness and generalization persist in graph-based vulnerability detection systems. Consequently, recent studies have begun exploring counterfactual data augmentation strategies to enhance both model robustness and explanation reliability .

Despite substantial progress by graph neural network (GNN)–based approaches for automated vulnerability detection, these models often suffer from limited robustness and poor generalization due to spurious correlations learned from training data.Such correlations can lead vulnerability detectors to rely on superficial code patterns rather than the true semantic causes of vulnerabilities, thereby limiting their reliability in practical software security analysis. To address this, Egea et al. introduced VISION [50], a framework designed to improve both the robustness and interpretability of graph-based vulnerability detection models through counterfactual data augmentation.

VISION generates counterfactual code samples that introduce minimal, semantic-preserving modifications to source code, enabling the model to distinguish between superficial correlations and the underlying structural causes of vulnerabilities. By leveraging large language models (LLMs) to generate these examples and representing source code as Code Property Graphs (CPGs) [16], the framework trains a GNN-based vulnerability detector capable of learning more robust and semantically meaningful representations of vulnerable code patterns. Additionally, it incorporates an explanation module that provides interpretable insights into the model's predictions, allowing developers to identify the specific code regions responsible for detected vulnerabilities.

Extensive experiments on the PrimeVul dataset, focusing on CWE-20 (Improper Input Validation) vulnerabilities, demonstrate that counterfactual augmentation substantially improves both robustness and predictive performance. Specifically, balanced training configurations incorporating counterfactual samples achieve superior accuracy, precision, recall, and F1-score compared to models trained solely on original data. Furthermore, embedding space analysis and attribution consistency evaluations reveal that counterfactual augmentation effectively mitigates spurious correlations and promotes more semantically consistent representations of vulnerable and benign code.

Nevertheless, the framework exhibits several notable limitations. First, its evaluation focuses on a single vulnerability category (CWE-20), limiting assessment across diverse vulnerability types. Second, counterfactual sample generation relies on LLMs without formal verification of semantic correctness. Third, VISION operates at the intra-program level, not modeling vulnerability propagation across software ecosystems or supply chain dependencies.Thus, VISION represents an important step toward robust and interpretable graph-based vulnerability detection.

2.2.5 Domain-Specific Applications

Xu et al. [51] introduced a graph-based framework for smart contract vulnerability detection that leverages Graph Neural Networks (GNNs) to overcome the limitations of rule-based static analyzers and conventional machine learning techniques . Their approach addressed traditional methods' reliance on handcrafted patterns and their limited scalability in multi-vulnerability detection scenarios. The framework modeled smart contracts as graph representations—derived from EVM bytecode and Solidity source code using Control Flow Graphs (CFGs) and Abstract Syntax Graphs (ASGs)—enabling structural learning of vulnerability patterns directly from program representations. Evaluation on 25,350 smart contracts covering eight vulnerability types achieved strong detection performance, reaching 95.78% accuracy and 93.80% F1-score, underscoring the effectiveness of graph-based learning for blockchain security anal-

ysis. Nevertheless, the framework exhibits notable limitations: it focuses on predefined vulnerability categories and does not model vulnerability propagation or zero-day risks across interconnected software ecosystems .

Przymus and Durieux, in their study ”Wolves in the Repository” [52], presented a comprehensive software engineering analysis of the XZ Utils supply chain attack (CVE-2024-3094). Their work addressed socio-technical vulnerabilities in open-source development ecosystems, highlighting the limitations of traditional vulnerability research that often focuses solely on code-level flaws. Instead, they examined governance manipulation and long-term social engineering strategies targeting project maintainers. By systematically analyzing Git commit histories, mailing-list discussions, and GitHub activity logs, the study meticulously reconstructed the multi-year infiltration process. This reconstruction revealed how an attacker gradually established credibility before ultimately injecting a malicious backdoor into the XZ Utils release pipeline. The analysis underscored the critical role of governance manipulation and social dynamics in enabling software supply chain compromises, establishing the attack as a significant example of socio-technical security risks in open-source ecosystems. Nevertheless, the study has notable limitations: it relies on a single case study and publicly available artifacts, which restricts the generalizability of its findings and may overlook covert attacker coordination. Thus, while this work significantly contributes to software supply chain security research, future studies must develop scalable mechanisms to detect anomalous behavioral patterns across open-source development communities .

Xia et al. introduced VDGraph, a graph-based framework for analyzing vulnerabilities in software supply chains [53]. It integrates Software Bill of Materials (SBOM) and Software Composition Analysis (SCA) data into a unified dependency graph model. Their approach overcomes the limitations of traditional vulnerability assessment techniques, which analyze dependency information and vulnerability reports independently, often leading to fragmented security visibility. By modeling software projects as dependency graphs that represent components and their vulnerability relationships, VDGraph enables systematic analysis of vulnerability propagation across transitive dependencies. Evaluation on 21 real-world Java projects, using vulnerability data from CVE [7] and GitHub Security Advisories (GHSA) [8], demonstrated that many vulnerabilities originate from deeply nested dependencies rather than direct project components. This underscores the importance of dependency-aware analysis in modern software supply chains. However, the framework has notable limitations: it relies on structural graph analysis without incorporating machine learning mechanisms to predict unknown or zero-day vulnerabilities. Therefore, while VDGraph marks an important step toward improving vulnerability visibility in software supply chains, future research should develop adaptive graph learning models [4] capable of predicting vulnerability propagation risks .

Jian et al. introduced a graph-based framework for vulnerability propagation analysis in software supply chains using Graph Neural Networks (GNNs) [55]. Their approach addresses the limitations of traditional vulnerability detection techniques, which primarily identify isolated software weaknesses without analyzing propagation risks across dependency networks [6]. By modeling the software supply chain as a dependency graph [13]—where nodes represent software components and edges capture dependency relationships—the framework enables structural analysis of vulnerability transmission across interconnected software ecosystems. Experimental results demonstrated that this GNN-based approach effectively identifies vulnerability propagation paths and improves risk assessment in large-scale software supply chain environments. Nevertheless, the framework exhibits notable limitations: it focuses on known vulnerability propagation patterns and does not explicitly address zero-day vulnerability prediction in evolving software supply chains [].

Table 2.3: Domain-Specific Vulnerability Detection Approaches

Study	Domain	Vulnerability Type	Approach	Dataset / Evaluation	Key Strength	Limitations
Xu et al. [51]	Blockchain / Smart Contracts	Reentrancy, integer overflow, access control flaws	Graph-based learning using GNNs with CFG and ASG representations derived from EVM bytecode and Solidity code.	25,350 smart contracts covering eight vulnerability types.	Learns structural vulnerability patterns from smart contract graphs; reports 95.78% accuracy and 93.80% F1.	Limited to predefined vulnerability types; does not model vulnerability propagation or zero-day threats.
Przymus and Durieux [52]	Socio-technical supply chain security	Maintainer compromise and malicious code injection in the XZ Utils attack.	Software engineering analysis of repository history, mailing lists, and developer activity.	Case study of the XZ Utils attack, CVE-2024-3094.	Reveals socio-technical attack strategies targeting governance and maintainer trust.	Single case study; limited generalizability for automated detection.
Xia et al. [54]	Software Supply Chain	Dependency vulnerabilities across transitive libraries.	Graph-based dependency analysis integrating SBOM and SCA data.	21 real-world Java projects using CVE and GHSA vulnerability data.	Enables analysis of vulnerability propagation through dependency graphs.	Does not incorporate machine learning for predicting unknown vulnerabilities.
Jian et al. [55]	Software Supply Chain	Vulnerability propagation across dependency networks.	Graph Neural Network model for analyzing vulnerability propagation paths.	Experimental evaluation on supply chain dependency graphs.	Identifies vulnerability propagation paths and improves risk assessment.	Focuses on known vulnerability patterns; limited capability for zero-day prediction.

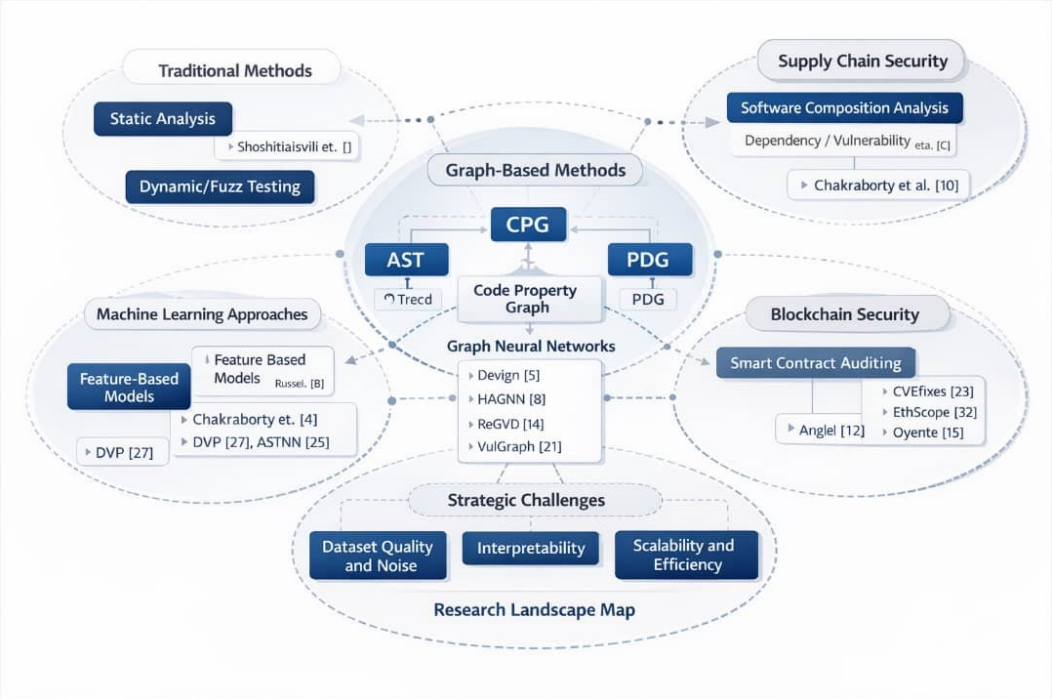


Figure 2.1: Research landscape of vulnerability detection techniques.

This figure illustrates the relationships among traditional analysis methods, machine learning approaches, and graph-based learning frameworks. It also depicts their applications in software supply chain security and blockchain-based smart contract analysis, while highlighting key research challenges such as dataset quality, interpretability, and scalability.

2.3 Comparative Analysis and Discussion

This section presents a comparative analysis of the representative vulnerability detection studies reviewed in this chapter, highlighting their graph representations, datasets, key strengths, limitations, and emerging research directions. Table 2.4 provides a comparative overview of selected graph-based vulnerability detection frameworks.

Table 2.4: Comparative Summary of Representative Graph-Based Vulnerability Detection Frameworks

Framework	**Graph Representation**	**Dataset**	**Key Strength**	**Limitations**
Devign [17]	Code Property Graph (AST + CFG + PDG)	Devign dataset (Linux, FFmpeg, QEMU, Wireshark)	Captures semantic relationships in source code using GNNs	High computational complexity and limited explainability
VulPathFinder [53]	Code Property Graph (CPG)	SARD dataset	Provides explainable vulnerability localization through vulnerability path analysis	Limited to intra-program vulnerability detection
HAGNN [23]	Inter-Procedural Abstract Graph (IPAG)	NVD, SARD	Models heterogeneous program relationships including function calls and data flows	Does not consider software ecosystem dependencies
ANGEL [18]	Hierarchical CPG + Graph Transformer	Devign, Reveal, Big-Vul	Improves scalability for large code graphs using hierarchical graph refinement	Focuses mainly on single-program vulnerability detection
VDGraph [54]	Software Dependency Graph	Java projects with CVE and GHSA	Enables vulnerability propagation analysis in software supply chains	Lacks predictive capability for unknown vulnerabilities
Jian et al. [55]	Dependency Graph + GNN	Supply chain vulnerability datasets	Models vulnerability propagation across dependency networks	Focuses mainly on known vulnerabilities
Xu et al. [51]	CFG + Abstract Syntax Graph (ASG)	Smart contract dataset (25k contracts)	Effective detection of blockchain vulnerabilities using graph learning	Limited coverage of evolving attack patterns

The reviewed literature reveals a clear transition from traditional program analysis techniques toward data-driven and graph-based learning frameworks. These frameworks are better suited for modeling the complex structural and semantic relationships inherent in modern software systems, reflecting the growing demand for scalable and intelligent vulnerability detection mechanisms to address the increasing complexity of contemporary software ecosystems.

Traditional vulnerability detection techniques such as static analysis, dynamic analysis, and fuzz testing remain foundational in software security analysis. Static analysis enables early vulnerability discovery without executing programs, allowing developers to detect potential weaknesses early in the development lifecycle. Dynamic analysis complements this by observing program behavior during runtime, identifying vulnerabilities that manifest only during execution. Fuzz testing further enhances discovery by generating large volumes of random or malformed inputs to trigger unexpected program behavior and expose memory-related flaws. Despite their effectiveness, these techniques often struggle with scalability and semantic complexity when applied to modern large-scale software systems.

Specifically, static analysis tools frequently produce high false-positive rates due to their limited contextual understanding of program semantics, while dynamic analysis and fuzz testing are constrained by execution coverage and input generation strategies.

To address these limitations, machine learning-based vulnerability detection approaches have been proposed. Early methods typically relied on token-level representations of source code or manually engineered features. While these approaches improved automation in vulnerability detection, they often failed to capture deeper structural relationships between program components. Consequently, they often struggle to identify complex vulnerability patterns arising from interactions among multiple program structures.

Graph-based vulnerability detection approaches represent a major advancement in this domain, modeling source code as structured graphs to capture relationships between program elements. Graph representations like Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), and Program Dependence Graphs (PDGs) explicitly model syntactic and semantic dependencies within software programs. These can be further unified into Code Property Graphs (CPGs), enabling comprehensive program analysis through integrated structural representations. When combined with Graph Neural Networks (GNNs), these representations enable learning models to capture hierarchical semantic relationships within code. Frameworks like Devign [5] and ANGEL [9] demonstrate the effectiveness of graph-based learning in improving vulnerability detection performance through structural program analysis.

Despite their advantages, graph-based vulnerability detection frameworks face several challenges. A primary limitation is computational scalability, as constructing and processing large program graphs demands substantial computational resources. Furthermore, many existing frameworks focus solely on vulnerability detection within individual programs, neglecting dependency relationships across software ecosystems. This oversight is particularly critical in modern software supply chains, where vulnerabilities can propagate through complex dependency networks and third-party components. Approaches like VDGraph [10] and the vulnerability propagation framework by Jian et al. [12] attempt to mitigate this by modeling software dependencies with graph-based representations, enabling vulnerability propagation analysis across ecosystems.

Interpretability presents another significant challenge for graph-based vulnerability detection models. While deep graph learning models often achieve high detection accuracy, many frameworks offer limited explanations for classifying specific code segments as vulnerable. Although approaches like VulPathFinder [7] attempt to improve interpretability through vulnerability path analysis, explainable vulnerability detection remains an underdeveloped research area.

The quality of datasets also critically impacts the effectiveness of vulnerability detection techniques. Benchmark datasets like SARD, BigVul, and CVE-based collections [5], [7], [9], [14], [23] serve as valuable resources for evaluating vulnerability detection models. However, these datasets often contain noisy samples, inconsistent labeling, and limited representation of emerging vulnerability categories. Consequently, models trained on them may exhibit reduced generalization capabilities in real-world software environments.

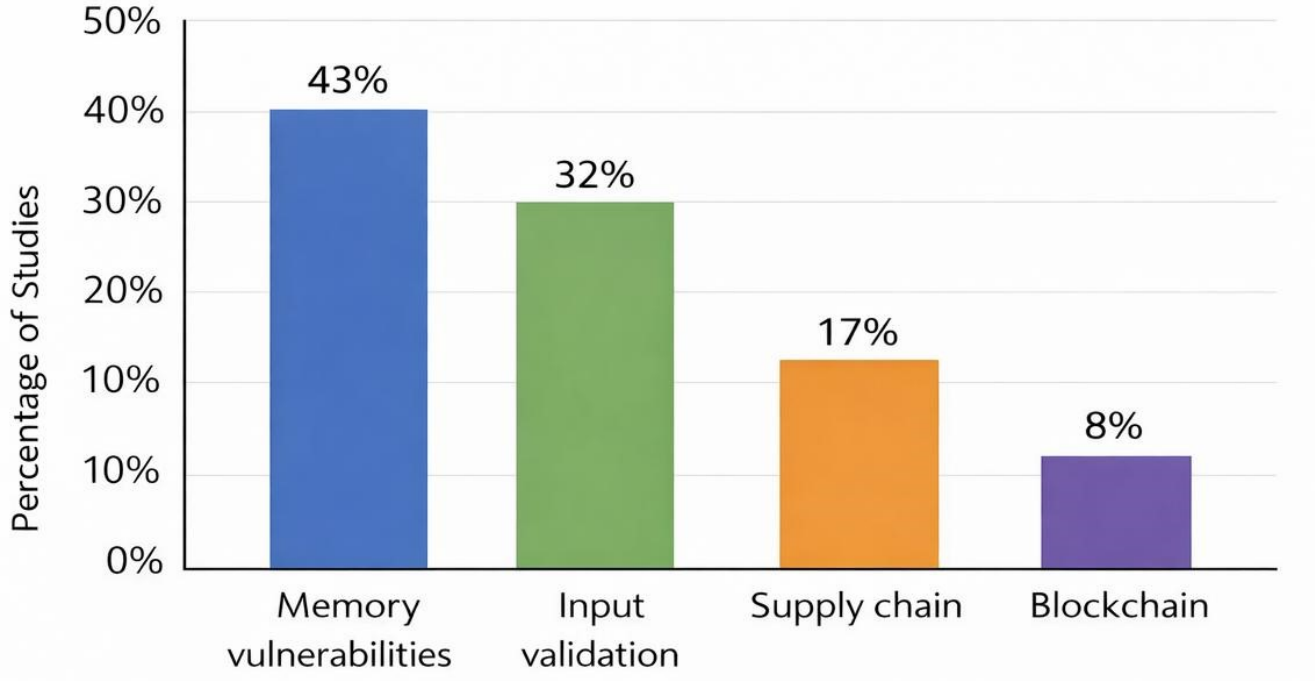
Beyond these challenges, vulnerability detection research is expanding beyond traditional program-level vulnerabilities. Emerging domains like software supply chain security and blockchain-based applications introduce new classes of vulnerabilities requiring specialized detection techniques. Specifically, in software supply chains, vulnerabilities can propagate through complex dependency relationships among interconnected software components, while in blockchain environments, they frequently arise from logical flaws within smart contract execution mechanisms.

Overall, this comparative analysis reveals several important research trends. First, graph-based learning frameworks consistently outperform feature-based methods for modeling complex semantic relationships in source code. Second, most existing vulnerability detection approaches are limited to intra-program analysis, rarely considering vulnerability propagation across software ecosystems. Third, explainable vulnerability detection remains an open challenge despite advances in deep learning-based approaches. Finally, emerging domains like software supply chain security and blockchain systems are receiving increased research attention due to the growing complexity of modern software infrastructures.

These observations highlight critical research challenges for future work. Specifically, future vulnerability detection frameworks should prioritize improving scalability for large program graphs, enhancing interpretability for practical vulnerability remediation, and developing higher-quality datasets that represent diverse vulnerability categories across evolving software ecosystems. Addressing these

challenges is essential for building robust and trustworthy vulnerability detection systems that support secure software development in increasingly interconnected computing environments.

Figure 3 illustrates the distribution of research focus across different vulnerability categories considered in this chapter



Distribution of research focus across vulnerability categories based on the representative studies analyzed in this chapter.

Figure 3 illustrates the distribution of research focus across the vulnerability categories examined in this chapter based on the representative studies analyzed in Sections 3.1–3.3. The reviewed literature reveals a comparable concentration on memory-related vulnerabilities, input validation issues, and software supply chain security, each representing a substantial portion of the analyzed studies. In contrast, blockchain-related vulnerabilities receive noticeably less attention within the current body of research.

This distribution reflects the historical emphasis of software security research on traditional program-level vulnerabilities, particularly memory safety errors and input validation flaws, which have long been recognized as major sources of software exploitation. At the same time, the growing presence of studies focusing on software supply chain security highlights the increasing awareness of dependency-based risks in modern software ecosystems.

Nevertheless, the relatively limited number of studies addressing blockchain security indicates an emerging research direction that remains insufficiently explored. Given the rapid expansion of decentralized applications and smart contract platforms, this observation suggests the need for more advanced vulnerability detection approaches specifically designed for blockchain-based software systems.

Chapter 3

Research Methodology

3.1 Introduction

In this chapter, we delineate the methodological framework. Specifically, we define the research design, data collection strategy, preprocessing procedures, feature engineering process, graph construction approach, proposed model architecture, training and validation strategy, evaluation metrics, implementation tools, and key methodological assumptions. Figure 3.1 presents the primary methodological flow of the proposed framework.

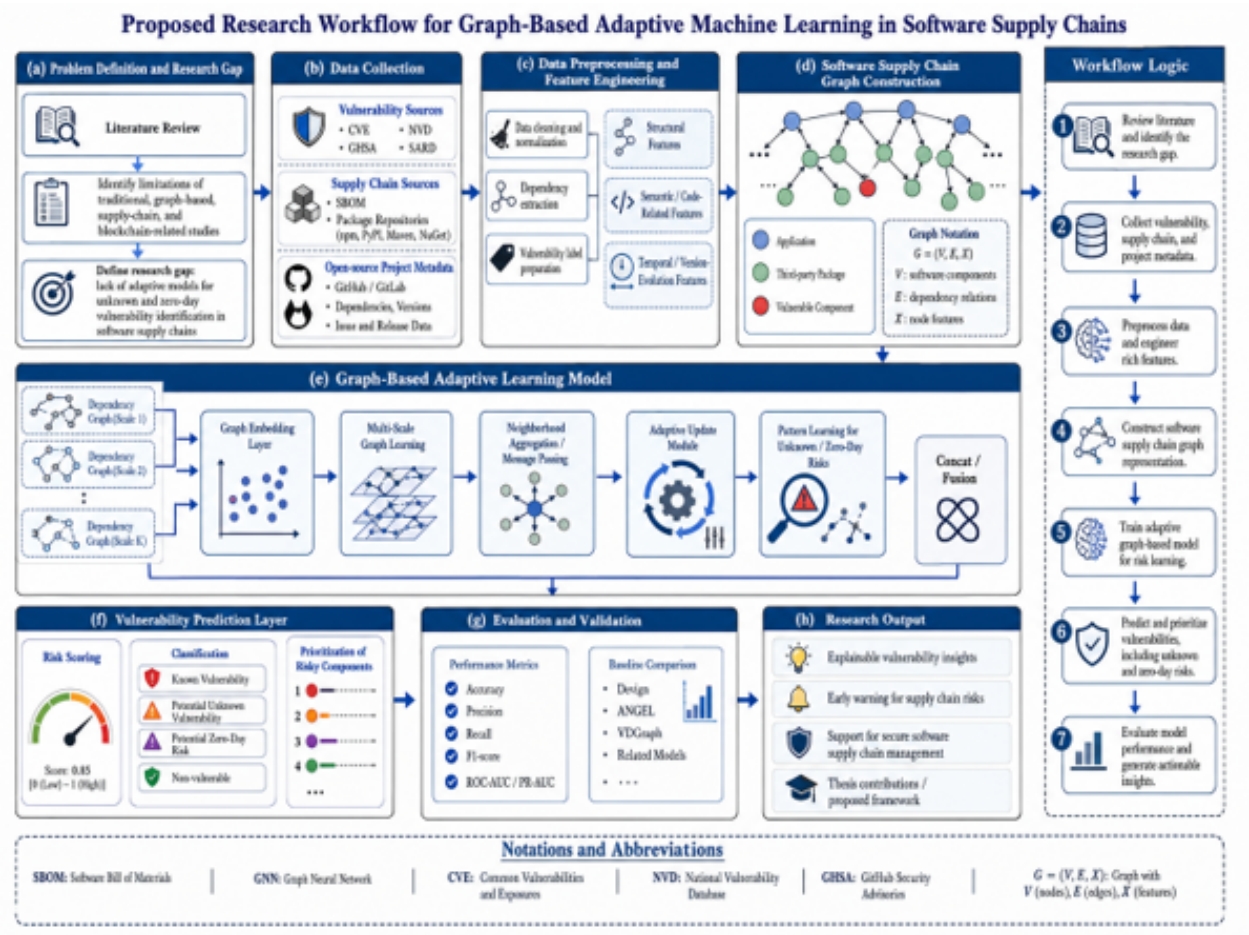


Figure 3.1: Research workflow of the proposed graph-based adaptive machine learning framework for identifying unknown and zero-day vulnerabilities in software supply chains.

As shown in Fig. 3.1, the methodology converts raw vulnerability and software supply chain data into structured graph representations. The proposed model then learns dependency-related patterns, updates its knowledge when new data becomes available, and produces outputs such as vulnerability labels, risk scores, and explainable indicators for analyst interpretation.

3.2 Research Design

This research adopts a quantitative experimental methodology. The quantitative aspect appears in the use of measurable variables, numerical feature matrices, graph-based node representations, and performance metrics such as accuracy, precision, recall, F1-score, AUC-ROC, false positive rate, and false negative rate. The experimental aspect appears in training and testing the proposed model and comparing its performance with baseline approaches.

The independent variables include component-level features, dependency-level features, vulnerability-related attributes, version-based indicators, temporal features, and graph-based structural features. The dependent variable is the predicted vulnerability status or risk score of a software component or dependency path. The main hypothesis is that graph-based adaptive learning can capture structural and contextual dependency patterns that improve vulnerability risk detection in software supply chains.

The study does not rely on questionnaires, interviews, or qualitative data collection. Instead, it focuses on computational experimentation using structured vulnerability and software supply chain data. The purpose is to evaluate whether a dependency-aware graph model can support detection of known vulnerable components and indicate suspicious dependency structures that may represent unknown or potential zero-day vulnerability risks.

3.3 Data Collection

Data collection is the first practical stage of the proposed methodology. It provides the raw input required for dataset construction, dependency graph generation, feature extraction, and model training. Since software supply chain risks involve vulnerabilities, packages, versions, dependencies, and project activity, the data will be collected from selected vulnerability databases, package registries, dependency files, SBOM records, and open-source repository metadata.

To keep the experimental scope feasible for a master's thesis, the primary implementation may focus on one ecosystem, such as PyPI, together with vulnerability records from NVD and OSV. Other sources, such as GitHub metadata, SBOM files, or additional package registries, can be used as supporting or optional sources depending on dataset availability and implementation scope.

Table 3.1: Data sources, collection methods, extracted fields, and intended usage

No.	Data Source	Collection Method	Extracted Data	Usage
1	Vulnerability records	NVD API 2.0, OSV API, GHSA advisory records, and CVE identifiers where available [1].	Vulnerability ID, affected package, affected version range, severity score, description, disclosure date, and CWE/CVE metadata.	Generate binary labels, where 1 = known vulnerable and 0 = non-vulnerable where applicable, and vulnerability-related features.
2	Package registry data	PyPI JSON API as the main ecosystem source; npm, Maven Central, or NuGet may be considered in extended experiments.	Package name, versions, release dates, maintainers, dependencies, and update frequency.	Build package metadata and support component-level and version-based analysis.
3	Dependency specification files	Extraction from requirements.txt, pyproject.toml, setup.py, package.json, pom.xml, or similar files depending on the ecosystem.	Direct dependencies, package requirements, version constraints, and dependency versions.	Construct direct dependency edges and derive transitive dependency paths.
4	SBOM records	Existing or generated SBOM documents using standard SBOM formats when available.	Component names, component versions, licenses, suppliers, and dependency relationships.	Validate component inventory and support graph construction.
5	Repository metadata	GitHub REST API, GitLab metadata, or repository mining tools.	Commits, issues, releases, contributors, repository age, and maintenance indicators.	Capture project activity, update behavior, and temporal features.

The collected records will be organized into a unified dataset before subsequent processing. Vulnerability records will support known vulnerability labeling; package registry data will describe software components; dependency files and SBOM records will provide graph edges; and repository metadata will contribute contextual and temporal indicators.

3.4 Data Preprocessing

Data preprocessing transforms the collected records into a consistent format suitable for feature engineering, graph construction, and model training. This stage is required because vulnerability databases, package registries, dependency files, SBOM records, and repository metadata may contain missing values, duplicate records, inconsistent package names, non-standard version formats, and incompatible structures.

Table 3.2: Data preprocessing procedures and expected outputs

No.	Preprocessing Step	Procedure	Example of Error	Purpose
1	Data cleaning	Remove irrelevant, incomplete, invalid, or corrupted records.	Records without package name or vulnerability identifier.	Improve dataset quality and reduce noise.
2	Duplicate removal	Identify and remove repeated package, vulnerability, or dependency records.	Repeated CVE entry linked to the same package version.	Prevent duplicated samples and biased model training.
3	Missing value handling	Handle missing package names, versions, severity scores, release dates, or dependency fields through removal or imputation when appropriate.	Missing release date or missing severity score.	Maintain dataset consistency and reliability.
4	Package name normalization	Standardize package names, capitalization, namespaces, and ecosystem-specific identifiers.	Django, django, and PyPI:django represented differently.	Ensure correct mapping between packages, vulnerabilities, and dependencies.
5	Version parsing and normalization	Parse and normalize version strings and affected version ranges using version parsing tools such as packaging.version in Python.	1.2.3-alpha, v2.0, >=1.0,<2.0.	Support accurate vulnerability-to-version mapping.
6	Vulnerability-to-package mapping	Link vulnerability records to affected packages and versions.	CVE record lists a product name that differs from the registry package name.	Generate known vulnerability labels and vulnerability-related features.
7	Data integration	Merge vulnerability records, package metadata, dependency relationships, SBOM data, and repository metadata.	Different schemas across NVD, OSV, PyPI, and repository metadata.	Construct a unified dataset for graph construction and model training.

After preprocessing, package names and versions will be standardized to support accurate mapping between vulnerability records and software components. Dependency relationships will be prepared as graph edges, while vulnerability labels and metadata will be assigned as node-level or edge-level attributes for later model training.

3.5 Feature Engineering and Selection

Feature engineering converts the cleaned and integrated dataset into measurable variables that describe software components, dependency relationships, vulnerability history, version behavior, graph structure, and project activity. These features form the feature matrix required by the graph-based learning model.

Feature engineering converts the cleaned and integrated dataset into measurable variables that describe software components, dependency relationships, vulnerability history, version behavior, graph structure, and project activity. These features form the feature matrix required by the graph-based learning model.

Table 3.3: Feature categories, examples, and calculation methods

No.	Feature Category	Extracted Features	Calculation / Formula Example	Purpose
1	Component-level features	Package name, version, ecosystem, license, and package age.	`package_age = current_date - first_release_date`	Describe each software component.
2	Dependency-level features	Number of direct dependencies, transitive dependencies, and dependency depth.	`dependency_depth` = longest dependency path from a package node	Capture dependency structure.
3	Vulnerability-related features	Known vulnerability label, severity score, affected versions, and vulnerability count.	`known_vuln_flag = 1` if package-version matches an affected version range; otherwise `0` when labeled safe.	Represent known vulnerability indicators.
4	Version-based features	Release date, version age, update frequency, and outdated status.	`update_frequency = number_of_releases / package_age_months`	Capture package evolution and update behavior.
5	Repository activity features	Commits, issues, releases, contributors, and maintenance activity.	`maintenance_activity = commits_last_year / repository_age_months`	Quantify project activity and maintenance behavior.
6	Graph-based features	Node degree, centrality, dependency position, and neighbor risk.	`neighbor_risk` = average risk score of adjacent nodes.	Support graph learning and risk propagation analysis.

The extracted features will be converted into a feature matrix X, where each row represents a software component or component version and each column represents a selected feature. This matrix will be combined with dependency relationships to construct the software supply chain graph.

3.6 Graph Construction

Graph construction transforms the feature-enriched dataset into a graph-based representation. The software supply chain graph is defined as:

$$G = (V, E, X) \tag{3.1}$$

where V is the set of software supply chain entities, such as packages, versions, repositories, and vulnerability indicators; E is the set of edges representing relationships among entities, such as direct dependencies, transitive dependencies, version relationships, and vulnerability propagation paths; and X is the node feature matrix.

The first-order neighbourhood of a node v is given by:

$$N(v) = \{u \in V \mid (v, u) \in E\} \tag{3.2}$$

The framework further leverages K-hop neighbourhoods to capture indirect dependencies, as vulnerability risk may propagate through both direct and transitive dependencies. Following graph construction, dependency relationships serve as graph edges, extracted features are assigned to corresponding nodes, and known vulnerability information provides labels for supervised learning where available. The resulting graph dataset serves as input to the proposed model architecture.

3.7 Proposed Model Architecture

The proposed model architecture is designed to learn structural and contextual patterns from software components and dependency relationships. The architecture includes a graph input layer, graph representation learning layer, neighbourhood aggregation mechanism, adaptive learning module, and vulnerability prediction layer.

For implementation consistency, the experimental model will focus on one graph learning architecture, such as a Graph Convolutional Network (GCN). Other graph neural network variants, such as GraphSAGE or Graph Attention Networks, may be considered as baselines or future extensions, but the main methodology should remain centered on a clearly defined model.

$$H^{l+1} = \sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{l}W^{l}\right) \tag{3.3}$$

In Equation 3.3, H^l is the node representation matrix at layer l, $\tilde{A} = A + I$ is the adjacency matrix with self-loops, $\tilde{D}$ is the diagonal degree matrix of $\tilde{A}$, W^l is the trainable weight matrix at layer l, and σ is an activation function such as ReLU. This layer propagation equation enables each node to update its representation using information from neighboring dependency nodes.

$$\theta^{t+1} = \alpha \cdot \theta^{t} + (1 - \alpha) \cdot \nabla L^{t} \tag{3.4}$$

Equation 3.4 represents the adaptive update mechanism, where θ^t denotes the model parameters at time t, α is a smoothing factor, and ∇L^t is the gradient of the loss function computed from newly

available training data. This mechanism allows the model to update detection knowledge when new packages, versions, dependency relationships, or vulnerability records become available.

Table 3.4: Proposed model components and mathematical representation

No.	Model Component	Function	Mathematical Representation	Output
1	Graph input layer	Receives the software supply chain graph.	$G = (V, E, X)$, Equation 3.1.	Graph structure and node features.
2	Neighbourhood definition	Identifies dependency neighbors for each node.	$N(v)$, Equation 3.2.	First-order and K-hop dependency context.
3	Graph representation learning layer	Learns structural and contextual embeddings from nodes and edges.	GCN propagation, Equation 3.3.	Node embeddings.
4	Adaptive learning module	Updates detection knowledge using new packages, versions, dependencies, or vulnerability records.	Adaptive update, Equation 3.4.	Updated risk patterns.
5	Vulnerability prediction layer	Classifies components or dependency paths according to risk.	$\hat{y}_{vuln} = \text{sigmoid}(z_{vuln})$.	Vulnerability label or risk score.

The expected output of this stage is a graph-based adaptive model that learns from both node features and dependency relationships. The learned representations support identification of known vulnerable components and suspicious dependency structures that may indicate unknown or potential zero-day vulnerability risks

3.8 Training and Validation Strategy

Training and validation will be conducted after constructing the graph dataset and implementing the proposed architecture. The dataset will be divided into training, validation, and testing subsets. The training subset will be used to learn model parameters, the validation subset will be used for hyperparameter tuning and overfitting control, and the testing subset will be reserved for final evaluation on unseen data.

During training, the model will learn vulnerability-related patterns from node features, dependency edges, and known vulnerability labels. Since vulnerability datasets are often imbalanced, the loss function will include a class weight to reduce bias toward the majority class.

$$L = -\frac{1}{N}\sum_{i=1}^{N}[w \cdot y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)] \tag{3.5}$$

In Equation 3.5, N is the number of training samples, y_i is the true label, $\hat{y}_i$ is the predicted probability, and w is the class weight used to address class imbalance. A possible setting is $w = N_{\text{non-vuln}}/N_{\text{vuln}}$,

where $N_{\text{non-vuln}}$ and N_{vuln} denote the numbers of non-vulnerable and vulnerable samples, respectively.

Hyperparameters such as learning rate, number of graph layers, hidden dimensions, dropout rate, batch size, and number of training epochs will be tuned using the validation set. The final model will then be evaluated on the testing subset to assess generalization performance on unseen components and dependency structures.

3.9 Evaluation Metrics

Model evaluation measures the proposed framework's effectiveness in detecting known vulnerable components, distinguishing non-vulnerable components, and identifying suspicious dependency structures requiring further security analysis. Multiple metrics are employed because vulnerability detection requires balancing detection capability with analyst workload. Let TP denote true positives, TN true negatives, FP false positives, and FN false negatives. Table 3.5 summarizes the evaluation metrics used in this study.

Table 3.5: Evaluation metrics and their interpretation

No.	Metric	Purpose
1	$\text{Accuracy} = \dfrac{TP + TN}{TP + TN + FP + FN}$	Measure the overall correctness of model predictions.
2	$\text{Precision} = \dfrac{TP}{TP + FP}$	Measure how many components predicted as vulnerable are actually vulnerable.
3	$\text{Recall} = \dfrac{TP}{TP + FN}$	Measure how many actual vulnerable components are correctly detected.
4	$F1 = \dfrac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$	Provide a balanced measure between precision and recall.
5	$FPR = \dfrac{FP}{FP + TN}$	Measure how often safe components are incorrectly classified as vulnerable.

No.	Metric	Purpose
6	$FNR = \frac{FN}{FN + TP}$	Measure how often vulnerable components are incorrectly classified as safe.
7	$AUC = \int_0^1 TPR(FPR)\, d(FPR)$	Evaluate discrimination ability across different decision thresholds.

The evaluation results will be used to assess detection capability, generalization ability, and practical usefulness. False positives are important because they may increase analyst workload, while false negatives are critical because they may allow vulnerable components to remain undetected.

3.10 Implementation Environment and Methodological Considerations

The proposed framework will be implemented in Python, using PyTorch and PyTorch Geometric for graph neural networks [2], NetworkX for graph construction and analysis [3], and scikit-learn for baseline models and evaluation metrics [4]. Data preprocessing relies on pandas, NumPy, and packaging.version. The methodology assumes that package names and versions can be normalized to map vulnerability records to registry data, and that dependency relationships are extractable from registry metadata, dependency files, or SBOM records. The graph is treated as fixed within a defined training time window, with adaptive updates applied when new data becomes available. Limitations include dependence on the quality and availability of public vulnerability data, package metadata, and dependency information. Historical data may not represent undisclosed zero-day attacks. Additionally, incomplete dependency files, missing SBOM records, inconsistent version formats, and imbalanced vulnerability labels may reduce model performance and evaluation reliability.